JavaScript Core Concepts

A plain-text reference covering the fundamental ideas every JavaScript developer should know

What is JavaScript?

JavaScript is a lightweight, interpreted, and dynamically typed programming language. It was originally created to add interactivity to web pages but has since grown into a full-stack language used in browsers, servers, mobile apps, and even embedded systems. It follows the ECMAScript specification, which is updated annually to introduce new language features and improvements.

Variables and Scope

In JavaScript, variables can be declared using var, let, or const. The var keyword is function-scoped and was the original way to declare variables. However, it has confusing behavior around hoisting, which led to the introduction of let and const in ES6. The let keyword creates a block-scoped variable that can be reassigned. The const keyword also creates a block-scoped variable, but its binding cannot be reassigned after declaration. Both let and const are subject to the Temporal Dead Zone, meaning they cannot be accessed before their declaration in the code.

Closures

A closure is one of the most powerful and often misunderstood features of JavaScript. It occurs when a function retains access to its outer lexical scope even after that outer function has finished executing. In simple terms, a closure allows an inner function to remember the variables of its parent function. This behavior is the foundation for many patterns in JavaScript, including data privacy, factory functions, and the module pattern. Closures are created every time a function is created in JavaScript.

The this Keyword

The value of this in JavaScript depends entirely on how a function is called, not where it is defined. In a regular function, this refers to the object that called the function. In strict mode, this is undefined if the function is called without an object context. Arrow functions do not have their own this binding — they inherit it from the surrounding lexical scope. Understanding this is critical when working with object methods, event handlers, and class-based code.

Prototypes and Inheritance

JavaScript uses a prototype-based inheritance model rather than classical inheritance. Every object in JavaScript has an internal prototype link, and when a property is not found on an object, JavaScript automatically looks up the prototype chain until it finds the property or reaches null. ES6 introduced the class syntax, which is a cleaner way to write constructor functions and prototype assignments. However, under the hood, classes are still prototype-based. Understanding this distinction helps avoid confusion when debugging inheritance-related issues.

Asynchronous JavaScript

JavaScript is single-threaded, meaning it can only execute one operation at a time. To handle tasks like network requests or timers without blocking the main thread, JavaScript uses an asynchronous model built around the event loop. The event loop continuously monitors the call stack and task queues, pushing queued callbacks onto the stack when it is empty. This is what allows JavaScript to remain

responsive even while waiting for slow operations to complete.

Promises

A Promise is an object that represents the eventual completion or failure of an asynchronous operation. It can be in one of three states: pending, fulfilled, or rejected. Promises allow chaining with the then and catch methods, making it easier to handle sequences of asynchronous operations compared to nested callbacks, a problem historically known as callback hell. Promises form the foundation of modern async JavaScript and are used extensively in browser APIs and Node.js.

Async and Await

The async and await keywords, introduced in ES2017, provide a cleaner syntax for working with Promises. An async function always returns a Promise, and the await keyword can be used inside it to pause execution until a Promise resolves. This makes asynchronous code look and read like synchronous code, greatly improving readability and reducing the complexity of error handling. Under the hood, async and await are still built entirely on Promises.

The Event Loop in Detail

The event loop processes tasks from two queues: the microtask queue and the macrotask queue. Microtasks include Promise callbacks and are processed after every task, before the browser renders. Macrotasks include setTimeout and setInterval callbacks and are processed one at a time. This means that Promise callbacks will always run before setTimeout callbacks, even if both are already queued. Understanding this order of execution is essential for predicting the behavior of complex asynchronous code.

Modules

JavaScript modules allow developers to split code across multiple files and explicitly declare what each file exports and imports. The native ES Module system, introduced in ES6, uses the export and import keywords. Each module has its own scope, so variables defined in one module are not accessible in another unless explicitly exported. Modules support both named exports, where multiple values can be exported from a single file, and default exports, where one primary value is exported. Modern bundlers like Webpack and Vite build on top of this system to optimize code for production.

Conclusion

JavaScript is a language that rewards deep understanding. Its flexibility and dynamic nature make it powerful but also require developers to have a solid grasp of its core behaviors — scoping, closures, the prototype chain, and the asynchronous execution model. Mastering these fundamentals makes it significantly easier to learn any JavaScript framework or library built on top of them.